

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

Bases de Datos Avanzadas

CI-0141

Grupo 01

I Ciclo 2026

Proyecto Programado #1

Cliente Integrado de Motores de Bases de Datos Distribuidas

(Soporte de Consultas, Bitácora y Recuperación ante Fallos)

Docente:

M.Sc. Sleyter Angulo Chavarría

Estudiantes:

Josué Badilla Paredes - C00870

Giuliana Ortega Saborio - C15689

Aldo Cascante Gutierrez - C21767

Mario Cordero Aguilar - C22306

25 de mayo de 2026

Índice

1. Introducción	2
2. Arquitectura del cliente	2
2.1. Front-end	3
2.2. Back-end	3
3. Protocolos de recuperación	4
4. Diseño e implementación de la bitácora WAL	5
5. Resultados de las simulaciones de fallos y recuperación	6
5.1. No-Undo/No-Redo	6
5.2. No-Undo/Redo	7
5.3. Undo/No-Redo	7
5.4. Undo/Redo	8
6. Conclusiones y reflexiones	8
7. Referencias bibliográficas	9
Referencias	9

1. Introducción

Durante décadas, diversas organizaciones han confiado la gestión y almacenamiento de sus datos en el tradicional esquema de bases de datos relacional, gestionado empleando el lenguaje de consultas SQL. Sin embargo, la llegada de la era del big data y de transformación digital y tecnológica en diversos ámbitos de la vida, han abierto la puerta en la adopción de nuevos métodos y herramientas para la representación, gestión y almacenamiento de datos. Es aquí donde las bases de datos NoSQL surgen como una alternativa a los sistemas relacionales.

No existe un esquema de base de datos perfecto que se pueda emplear de forma efectiva y óptima todas las situaciones, sin embargo dependiendo del contexto o situación en concreto que se esté modelando, las características de cada esquema pueden ser aprovechadas de una mejor forma. Más aún cuando las bases de datos NoSQL al no poseer un esquema rígido, pueden comportarse y representar datos de una gran cantidad de formas.

Ante esto, ha proliferado el concepto de “persistencia políglota”, el cuál consiste en un patrón de arquitectura de software que utiliza múltiples tecnologías de bases de datos para satisfacer diferentes necesidades de almacenamiento dentro de un mismo sistema. A pesar de la variedad de opciones para emplear como bases de datos, estas tienen una características en común, que son propensas a sufrir errores inesperados. Ante este hecho, se diseñaron los protocolos de recuperación ante fallos(Arévalo, Santiago, y Andrade, 2017).

De esta forma se considera de gran relevancia contar con herramientas que permitan a las organizaciones emplear y gestionar diversos tipos de motores de bases de datos, para que así puedan emplear varias tecnologías aprovechando al máximo las ventajas de cada una de acuerdo al tipo de datos que estén empleando. Entonces, se plantea el desarrollo de un cliente integrado de motores de bases de datos distribuidas SQL y NoSQL, con soporte de consultas, bitácora y recuperación ante fallos.

2. Arquitectura del cliente

El cliente integrado consiste en una aplicación web full-stack. El front-end cumple la función de interfaz de usuario, mientras que el back-end la función de módulo de procesamiento y gestión de conexiones con motores de bases de datos para realizar en ellos consultas y operaciones enviadas por el usuario desde el front-end.

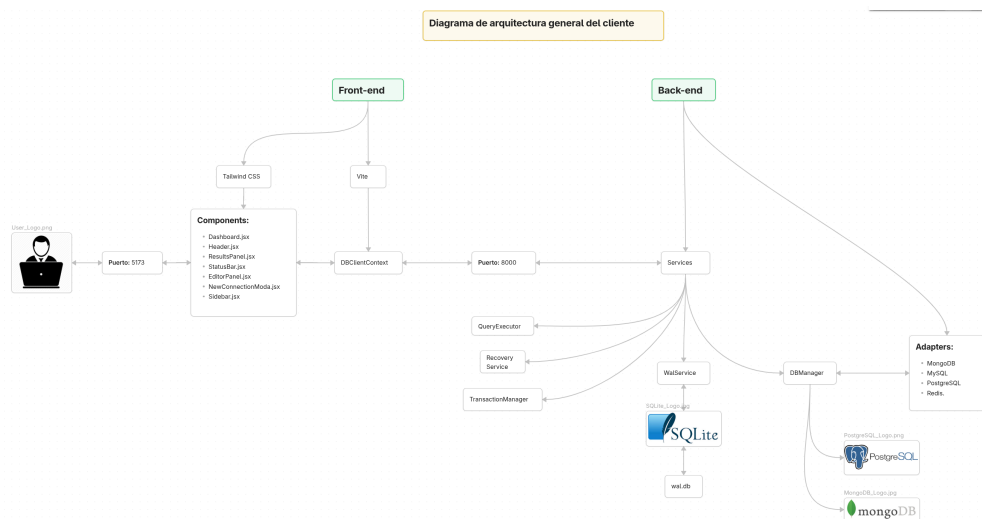


Figura 1: Diagrama de arquitectura general del cliente.

2.1. Front-end

El front-end está construido en la biblioteca de JavaScript React, se emplea Vite como herramienta de build y Tailwind CSS como framework de estilos. El código fuente del proyecto de front-end sigue una estructura común de organización, donde se emplea un directorio distinto para las partes de recursos (assets), componentes (components), configuración (config) y contextos (contexts). De forma predeterminada, el cliente utiliza el puerto 5173 para poder renderizarlo de forma local en cualquier navegador web moderno.

Para conectarse al back-end, se emplea un **DBClientContext**, el cuál consiste en una implementación específica de React Context diseñada para compartir y administrar una instancia de cliente de base de datos en la aplicación web. Su función principal es guardar el estado global de la aplicación (conexiones, transacciones, logs) y conectarse con un servidor de back-end a través de varias APIs para que cualquier componente del front-end pueda emplear dicha información sin tener que estar pasándola entre varios elementos intermedios.

Sus funcionalidades se pueden dividir en 5 categorías:

- Gestión de conexiones a bases de datos.
- Editor y ejecución de consultas a bases de datos.
- Control de transacciones y WAL (Write-Ahead Log).
- Simulación de fallos y recuperación.
- Consola del sistema y estado de salud.

2.2. Back-end

El back-end está implementado utilizando el lenguaje de programación Python, empleando el framework de desarrollo de APIs FastAPI. El back-end utiliza el puerto 8000 de forma predeterminada para comunicarse con el front-end.

Para realizar todas las operaciones relacionadas a la comunicación y procesamiento de consulta provenientes del front-end, el back-end cuenta con una capa de servicios, donde cada uno tiene un propósito específico. Estas divisiones de responsabilidades implican en un diseño con alta cohesión y baja acoplamiento.

Servicios del back-end:

- **DBManager**: se encarga de la gestión con las bases de datos, contiene funciones para gestionar las conexiones y los adaptadores para permitir la comunicación con ellas.
- **QueryExecutor**: verifica, hace parsing, y ejecuta las consultas enviadas desde el front-end.
- **RecoveryService**: se encarga de la ejecución de las operaciones de los 4 protocolos de recuperación ante fallos y de la simulación de fallos.
- **TransactionManager**: gestiona las transacciones realizadas considerando cada uno de sus posibles estados.
- **WalService**: gestiona la bitácora de tipo WAL.

Recapitulando, el cliente debe ser capaz de conectarse a varios motores de bases de datos, donde cada uno tiene su forma de recibir datos, además de una estructura específica para sus consultas. Para solucionar esta situación, se decidió emplear el patrón de diseño adaptador, donde cada motor tiene una clase adaptador que traduce y se ajusta a las especificaciones de cada uno. Este diseño es ampliamente abierto a la expansión, en caso de que se deseen agregar más motores. Actualmente se cuenta con adaptadores para MongoDB, MySQL, PostgreSQL y Redis.

3. Protocolos de recuperación

La recuperación basada en logs es uno de los métodos de recuperación más comúnmente empleados en las bases de datos modernas. Se mantiene un archivo de registro (diario) en un medio de almacenamiento estable que registra cada cambio (inserción, actualización o eliminación) realizado en una transacción antes de que se aplique a la base de datos en sí.

Si ocurre algún fallo, se definen 2 operaciones para tratar a los datos afectados:

- **Undo:** deshacer los cambios de una transacción fallida.
- **Redo:** rehacer los cambios de una transacción ya confirmada (committed).

En base a estas 2 operaciones, se definen 4 protocolos/algoritmos, cada uno trata con los datos afectados a su manera, al cambiar el orden y resultado de las operaciones de Undo y Redo definidas anteriormente. Estos 4 algoritmos especifican cómo el Manejador de Recuperaciones maneja las diferentes operaciones: begin-transaction, read, write, commit, abort y restart.

De acuerdo con el título de los algoritmos, éstos difieren de acuerdo a si tienen que rehacer y/o deshacer las actualizaciones hechas en la base de datos luego de la falla. Esta decisión depende de cuándo el buffer de la base de datos fue guardada en un almacenamiento permanente (Bertone, Ruscuni, Miatón, y Mauriello, 2001). Si fueron almacenados manera sincrónica durante la ejecución de la transacción, los cambios pueden tener que ser deshechos. Si en cambio, los buffers se almacenan sólo después de una escritura forzada por el Manejador de Recuperaciones, entonces puede no ser necesario deshacer. El rehacer una transacción depende de si los buffers fueron pasados sincrónicamente al commit, en cuyo caso no se necesita el redo, o asincrónicamente de acuerdo a las necesidades del manejador del buffer, en cuyo caso se puede requerir un redo (Bertone y cols., 2001).

Las siguientes son los protocolos de recuperación tratados en este trabajo:

1. No-Undo/No-Redo:

- La base de datos solo se actualiza después de que la transacción haga COMMIT.
- Mientras, los cambios se mantienen en el WAL.
- Si la transacción falla antes del COMMIT, no hay que hacer UNDO, ya que la base de datos nunca se modificó, ni REDO, por que no se guardó nada.

2. No-Undo/Redo:

- La base de datos sí se actualiza en disco antes del COMMIT mediante actualización inmediata.
- Si la transacción falla antes del COMMIT, no se aplica UNDO. Se deja que la base de datos quede "sucia", y se espera que el sistema detecte el fallo y la restaure desde un log.
- Si una transacción falla después del COMMIT, se utiliza REDO para volver a aplicar los cambios del log.

3. Undo/No-Redo:

- La BD sí se actualiza en disco antes del COMMIT mediante actualización inmediata.
- Si la transacción falla antes del COMMIT, se aplica UNDO utilizando la WAL, para deshacer los cambios de la base de datos.
- Si la transacción falla después del COMMIT, no se aplica REDO.

4. Undo/Redo:

- Es el enfoque más flexible ya que la base de datos puede actualizarse en disco antes o después del COMMIT.
- Si la transacción falla antes del COMMIT, se aplica UNDO para deshacer los cambios.
- Si la transacción ya hizo COMMIT, pero el sistema falla antes de que todos los cambios estén presentes en disco, se aplica REDO para rehacer cualquier cambio faltante registrado en el WAL.

4. Diseño e implementación de la bitácora WAL

Los sistemas de bases de datos no son ajenos a fallos, los cuales pueden ocurrir por diversas razones. Por lo tanto, se requiere de la utilización de mecanismos capaces de regresar una base de datos a un estado anterior en caso de que haya ocurrido algún fallo, evitando así posibles consecuencias negativas producto de tener datos erróneos y/o corruptos. Existen diversas técnicas de recuperación ante fallos, entre estas la basada en logs, shadow paging y checkpointing. En la implementación del cliente, nos vamos a basar en la primera técnica, ya que se cuenta con un módulo de bitácora de tipo Write-Ahead Log (WAL).

Para la recuperación ante eventuales fallos durante la ejecución de transacciones empleando el cliente, se implementó el servicio “WalService”. Esta define una serie de funciones para el registro y gestión de transacciones ocurridas en las diversas conexiones que se tengan con las bases de datos. A nivel general, este servicio consiste en una bitácora de tipo WAL, la cuál es una técnica empleada en sistemas de almacenamiento para asegurar la durabilidad y consistencia de sus transacciones (Vinci, 2024). La idea general detrás de WAL es registrar los cambios en un registro antes de aplicarlos al almacenamiento real. Esta bitácora, contiene un registro secuencial de todos los cambios realizados en una base de datos. Las transacciones no se consideran completas hasta que los cambios correspondientes se registren de forma segura en la bitácora (Vinci, 2024).

La bitácora implementada utiliza el motor de bases de datos relacional SQLite para gestionar los logs de forma persistente en el archivo “wal.db”. Además el servicio RecoveryService tiene comunicación directa con WalService a lo largo de sus funciones.

La bitácora está conformada por las siguientes tablas:

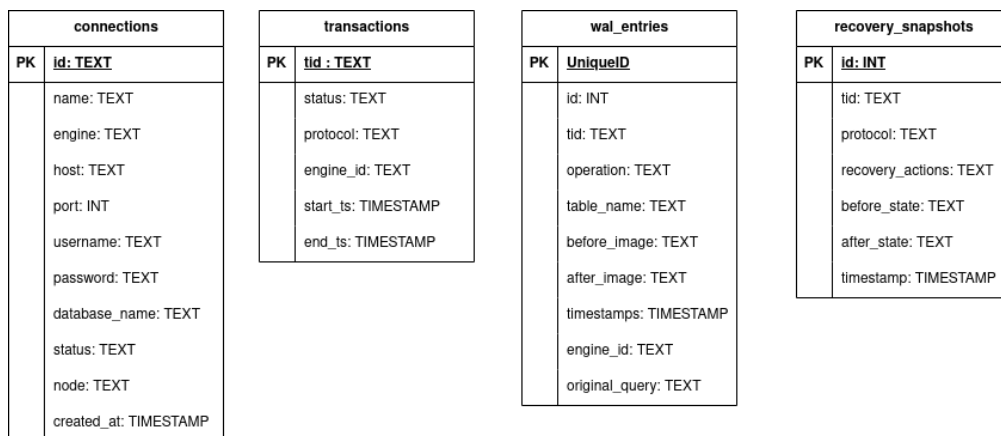


Figura 2: Diagrama del modelo relacional de las tablas de la base de datos “wal.db”.

Con estas tablas y sus respectivos campos, es posible almacenar de forma persistente toda la información de las diversas transacciones realizadas en cualquier motor de bases de datos empleado para así poder recuperarse ante fallos efectivamente.

5. Resultados de las simulaciones de fallos y recuperación

En el cliente, al iniciar una transacción esta queda activa hasta un final explícito vía una directiva ROLLBACK o COMMIT, o hasta que se presente un error, escenario en el cual brillan los protocolos de recuperación implementados. Para simular estos escenarios se implementó el llamado a un api (accesible desde el cliente) el cual simula un fallo en una transacción abierta, provocando el comportamiento del protocolo aplicado para dicha transacción.

Para ejemplificar estos comportamientos, se mostrará una tabla con 4 filas en el motor de postgresSQL. Estos datos se utilizaran en transacción donde sus valores serán modificados, en uno se simula el fallo y en el otro se llega al COMMIT, tras esto se intenta la recuperación y mostrará el resultado según el método.

```
BEGIN;

UPDATE students
SET grade = 0
WHERE name = 'Aldo';

SELECT * FROM students;
```

Ejecutar

ROLLBACK

COMMIT

Resultados	Bitácora (WAL)	Recuperación	Consola
1		Josue	100
2		Giuliana	100
3		Mario	100
0		Aldo	100

Figura 3: Captura de pantalla mostrando el estado inicial de los datos previo a las pruebas.

5.1. No-Undo/No-Redo

Resultados	Bitácora (WAL)	Recuperación	Consola
TID: TXN-0011 Protocolo: No-Undo/No-Redo Estado: completed Antes: Transaction TXN-0011: status=ABORTED, 2 write operation(s) in WAL Después: Recovery complete. 0 operation(s) applied. [No-Undo/No-Redo] Transaction did not commit – no-steal policy means dirty pages never reached disk. Nothing to undo.			

Figura 4: Captura de pantalla mostrando detalles y resultado de la recuperación tras simulación de fallo bajo No-Undo/No-Redo.



Figura 5: Captura de pantalla mostrando detalles y resultado de la recuperación tras COMMIT bajo No-Undo/No-Redo.

5.2. No-Undo/Redo

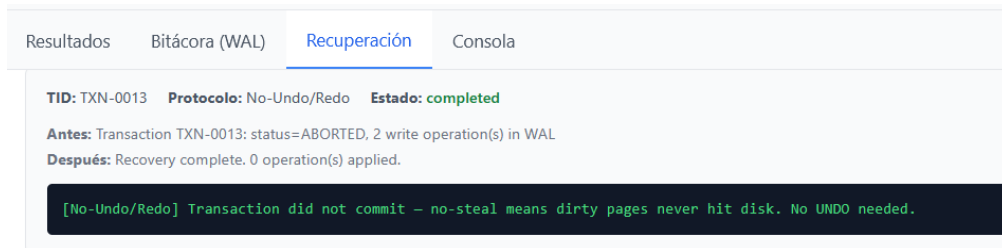


Figura 6: Captura de pantalla mostrando detalles y resultado de la recuperación tras simulación de fallo bajo No-Undo/Redo.



Figura 7: Captura de pantalla mostrando detalles y resultado de la recuperación tras COMMIT bajo No-Undo/Redo.

5.3. Undo/No-Redo

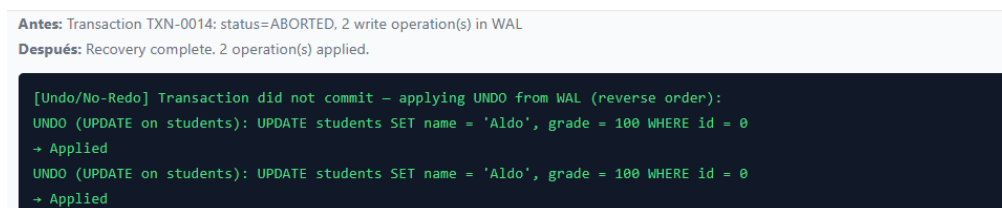


Figura 8: Captura de pantalla mostrando detalles y resultado de la recuperación tras simulación de fallo bajo Undo/No-Redo.


```
Antes: Transaction TXN-0022: status=COMMITTED, 2 write operation(s) in WAL
Después: Recovery complete. 0 operation(s) applied.

[Undo/No-Redo] Transaction committed – force policy guarantees all pages were on disk at commit. No REDO needed.
```

Figura 9: Captura de pantalla mostrando detalles y resultado de la recuperación tras COMMIT bajo Undo/No-Redo.

5.4. Undo/Redo

```
Antes: Transaction TXN-0015: status=ABORTED, 2 write operation(s) in WAL
Después: Recovery complete. 2 operation(s) applied.

[Undo/Redo] Transaction did not commit – applying UNDO (reverse order):
UNDO (UPDATE on students): UPDATE students SET name = 'Aldo', grade = 100 WHERE id = 0
→ Applied
UNDO (UPDATE on students): UPDATE students SET name = 'Aldo', grade = 100 WHERE id = 0
→ Applied
```

Figura 10: Captura de pantalla mostrando detalles y resultado de la recuperación tras simulación de fallo bajo Undo/Redo.

```
Antes: Transaction TXN-0021: status=COMMITTED, 2 write operation(s) in WAL
Después: Recovery complete. 2 operation(s) applied.

[Undo/Redo] Transaction committed – applying REDO:
REDO (UPDATE on students): UPDATE students SET grade = 0 WHERE name = 'Aldo'
→ Applied
REDO (UPDATE on students): UPDATE students SET grade = 0 WHERE name = 'Aldo'
→ Applied
```

Figura 11: Captura de pantalla mostrando detalles y resultado de la recuperación tras COMMIT bajo Undo/Redo.

6. Conclusiones y reflexiones

Existen múltiples formas de abordar las eventualidades de errores y necesidades para recuperar estados previos en bases de dato. Un WAL siendo una herramienta sumamente útil para asegurar la durabilidad y consistencia de las transacciones frente a posibles fallos.

También se concluye sobre lo útil que fue la utilización de Docker Compose para simular los nodos distribuidos donde se alojaban las instancias de las bases de datos. Sin duda su utilización fue más sencilla que emplear servicios en la nube, donde primero, hay que apegarse a las reglas y funcionamiento de cada proveedor, además de la continua conexión a Internet y por supuesto, el alto costo operativo que se tuvo que asumir. Siendo también de suma conveniencia para estandarizar el acceso a las bases, sin tener que preocuparse por las configuraciones de los equipos donde se fuera a ejecutar el servicio.

7. Referencias bibliográficas

Referencias

- Arévalo, J., Santiago, J., y Andrade, L. (2017). *Persistencia políglota caso de estudio: MongoDB y neo4j*. Descargado de <https://es.overleaf.com/articles/persistencia-poliglota-caso-de-estudio-mongodb-y-neo4j/dntbjwthzggm>
- Bertone, R., Ruscuni, S., Miatón, I., y Mauriello, A. (2001, May). Análisis de rendimiento y replicación en bases de datos distribuidas. *III Workshop de Investigadores en Ciencias de la Computación*. Descargado de <http://sedici.unlp.edu.ar/handle/10915/21602>
- Vinci, A. (2024, May). *Write ahead logs: But why?* Descargado de <https://medium.com/@vinciabhinav7/write-ahead-logs-but-why-494c3efd722d>